

Sección 2 — Repaso espaciado

Documentación técnica: motor SM-2, modelo de conocimiento, repaso previo y comparación con Leitner

1. Qué resuelve esta sección

La Sección 2 convierte la **bolsa de palabras** (lo que el usuario guarda mientras lee) en un sistema de estudio: cada palabra lleva un estado de **repetición espaciada** que decide cuándo volver a preguntarla. En producción la programación la gobierna **SM-2** (SuperMemo-2); **Leitner** se modela como cadena de Markov y se compara con SM-2 por simulación, fuera de producción. Además, un **modelo de conocimiento** estima qué palabras de una lectura el usuario probablemente no conoce y se las presenta como **repaso previo** antes del texto.

Decisiones de producto (fijadas con el usuario): SM-2 único en la UI para estandarizar la experiencia; calificación en 4 niveles estilo Anki; tarjeta palabra → traducción (reconocimiento); repaso previo saltable con tope por nivel (5 / 12 / 20 por capítulo).

2. Mapa de archivos

Pieza	Archivo	Qué contiene
Motor SM-2	src/engine/srs.js (+ .test)	Recurrencia SM-2, sesión de repaso, resumen; puro, tiempo inyectado
Modelo de conocimiento	src/engine/conocimiento.js (+ .test)	P(conocer) por palabra y selección de candidatas al repaso previo
Motor Leitner + Markov	src/engine/leitner.js (+ .test)	5 cajas, matriz de transición, estacionaria, tiempos de llegada, tasa
Tokenizador	src/engine/tokenizar.js	Regex Unicode compartido por Lector y modelo (mismas palabras)
Persistencia	src/engine/almacenamiento.js	localStorage: bolsa.v1, conocidas.v1, repasoPrevio.v1
UI sesión de repaso	src/secciones/repaso/Repaso.jsx	Ruta /repaso: cola vencidas+nuevas, tarjeta, 4 niveles
UI repaso previo	src/secciones/lectura/RepasoPrevio.jsx	Fichas antes del texto; SM-2 real para bolsa, estudio para nuevas
Botones compartidos	src/secciones/repaso/BotonesCalificacion.jsx	4 niveles con vista previa del intervalo resultante
Frecuencias del corpus	pipeline/frecuencias.py	Emite src/data/frecuencias.json (conteo por lema, vía léxico)
Simulación	simulacion/comparar.mjs	Alumno sintético; compara SM-2 vs Leitner sobre los motores reales

3. El motor SM-2

El estado SRS vive **dentro de cada entrada de la bolsa** bajo la clave srs; una entrada sin srs es una palabra nueva. El módulo es puro: el instante ahora se inyecta siempre como ISO string, así los tests y la simulación son deterministas y el mismo código corre en producción y en la simulación.

```
srs = { reps,      // aciertos consecutivos (n de SM-2)
        ef,        // factor de facilidad (suelo 1.3)
        intervalo, // días hasta el próximo repaso
        vencimiento, // ISO: cuándo vuelve a estar pendiente
        repasos, fallos } // estadística acumulada
```

3.1 Recurrencia

```
EF' = max(1.3, EF + 0.1 - (5-q)·(0.08 + (5-q)·0.02))
I(1) = 1, I(2) = 6, I(n) = round(I(n-1)·EF)      si q ≥ 3
I = 0 (repetir hoy), n = 0                        si q < 3
```

Los 4 botones de la UI mapean a la calidad q: **Otra vez**=2, **Difícil**=3, **Bien**=4, **Fácil**=5. Con q<3 la palabra vuelve al principio y se recicla al final de *la misma sesión* (vencimiento = ahora). EF se actualiza en toda respuesta, también en fallos, con suelo 1.3; nótese que q=4 deja EF exactamente igual (+0.1 - 1·0.10 = 0), por eso una palabra siempre calificada «Bien» mantiene EF 2.5.

3.2 Sesión

seleccionarSesion construye la cola: primero las vencidas (la más atrasada primero), después las nuevas por orden de incorporación con tope (10 por defecto). Una palabra sin traducción al español no es repasable (no hay reverso). El vencimiento es **24 horas rodantes** desde la calificación, no medianoche como Anki: si repasas a las 23:00 con intervalo 1, vence mañana a las 23:00; no hay reloj ni notificación, se compara el timestamp al abrir /repaso o la Bolsa.

4. El modelo de conocimiento

Estima P(conocer) por palabra (id = idioma: lemma), por casos:

Caso	P(conocer)	Racional
Marcada «ya la conocía»	0.95	El usuario lo afirmó (persistido en conocidas.v1)
En bolsa con estado SRS	$R = e^{-(t/S)}$	Retrievability: t = días desde el último repaso (= vencimiento - intervalo, recuperable del estado sin campos extra); S = max(intervalo, 0.5)
En bolsa sin repasar	0.2	La guardó precisamente porque no la conocía
Nunca vista	$\sigma((zipf - 3.0)/0.7)$	Prior logístico sobre la escala Zipf del corpus propio; zipf = $\log_{10}(\text{conteo}/\text{total} \cdot 10^6)$. Sin conteo: 0.1

Con el corpus actual (~30.300 tokens en alemán): «und» (993 apariciones) → zipf ≈ 4.5 → P ≈ 0.9; un hapax → zipf ≈ 1.5 → P ≈ 0.1. Las frecuencias las emite pipeline/frecuencias.py reutilizando el mapa forma→lemma del léxico (no necesita spaCy) y respetando los overrides por lectura (p. ej. «nahm» → annehmen).

4.1 Candidatas al repaso previo

Tokens únicos del cuerpo de la lectura, resueltos a lema (override de la lectura > léxico global), deduplicados por id, **solo con traducción al español** (excluye de paso los nombres propios), con P < 0.7, ordenados por P ascendente y recortados al tope del nivel: principiante 5, intermedio 12, avanzado 20 por parte. En *Die Verwandlung* la primera candidata que produce el modelo es «Ungeziefer» — exactamente el hapax que debe proponer.

5. El repaso previo (flujo de UI)

Al abrir una lectura en idioma extranjero, el Lector carga léxico y frecuencias (chunks dinámicos, fuera del bundle inicial), calcula las candidatas y, si hay alguna y no se mostró hoy para esa lectura, interpone

RepasoPrevio. Dos tipos de ficha: si la palabra ya está en la bolsa, tarjeta SM-2 *real* (voltear + 4 niveles, califica el estado de verdad); si es nueva, ficha de estudio con ambas caras y dos salidas: «Ya la conocía» (a conocidas.v1, no vuelve) o «A la bolsa» (entra con origen 'previo' y el SM-2 la programa). Siempre saltable («Ir al texto →»); al terminar o saltar se registra la fecha en repasoPrevio.v1 (máx. 1 vez/día/lectura).

Clave localStorage	Contenido
bolsa.v1	Array de entradas; el estado srs vive dentro de cada entrada
conocidas.v1	Array de ids marcados «ya la conocía»
repasoPrevio.v1	Mapa lecturald → fecha ISO del último previo mostrado
lecturas.completadas.v1	Progreso de lectura (Sección 1)

6. Leitner como cadena de Markov

Leitner (5 cajas, intervalos $I = [1, 2, 4, 8, 16]$ días): al acertar se sube de caja, al fallar se vuelve a la 1. Si la probabilidad de acierto p es constante, la sucesión de cajas es una cadena de Markov con matriz $P[i][i+1] = p$ y $P[i][1] = 1-p$ (techo en la 5). De ahí salen los resultados analíticos (implementados y testeados en engine/leitner.js):

```

π = πP                                distribución estacionaria (iteración de potencias)
E[repasos hasta caja 5] = Σ_{j=1..4} p^(-j)      (racha de 4 aciertos)
E[días hasta caja 5]: d_i = I_i + p·d_{i+1} + (1-p)·d_1, d_5 = 0
tasa en régimen = 1 / Σ_i π_i · I_i    repasos/día/palabra

```

Con $p = 0.9$: ≈ 5.24 repasos y ≈ 18.1 días para llegar a la caja 5, y en régimen una palabra consume ≈ 0.08 repasos/día. El detalle numérico y su contraste con la simulación están en **metricas-seccion2.pdf**.

7. La simulación comparativa

simulacion/comparar.mjs (Node, npm run simular) enfrenta los dos planificadores **sobre los motores reales del repo** con un alumno sintético de curva de olvido exponencial $P(\text{recordar tras } \Delta) = e^{(-\Delta/S)}$: al acertar S crece ($S \cdot g$), al fallar se contrae ($\max(S_0, 0.5 \cdot S)$). Mismo alumno (misma dificultad, S_0 y g por palabra) bajo cada planificador; 480 palabras (8/día durante 60 días), 150 días de horizonte, RNG determinista (mulberry32, semilla 42). Contabilidad fiel a cada método: un fallo en SM-2 cuesta 2 presentaciones (falla + reciclada en sesión, como la UI); en Leitner, 1 (baja de caja y espera). Salida: docs/datos-simulacion.json, que consume el generador de métricas.

8. Decisiones y trampas

Tema	Decisión / trampa
Identidad de palabra	id = idioma:lemma (normalizado); los verbos separables se identifican por su lema reconstruido (annehmen), no por la forma (nahm)
EF con «Bien»	q=4 deja EF sin cambio por diseño de SM-2; no es un bug
Vencimiento	24 h rodantes desde la calificación; no hay corte a medianoche
Navegación entre lecturas	El previo guarda parald (a qué lectura pertenece): un render intermedio al navegar mostraba fichas de la lectura anterior y rompía la cola
Peso del bundle	lexico.json (~324 KB) y frecuencias.json (~72 KB) son chunks de import dinámico; no entran en el bundle inicial
Calibración del alumno	Con $e^{(-t/S)}$, recordar al 90% exige $S \approx 9.5 \cdot \Delta$; $S_0 \sim U(6,16)$ días y $g \approx 2.2\text{--}3.6$ dan precisión de repaso ~80-90%, como los SRS reales
Python + umlauts	Siempre PYTHONUTF8=1 y ensure_ascii=False (regla del repo)

9. Cómo regenerar cada artefacto

```
python pipeline/frecuencias.py      # src/data/frecuencias.json
npm test                            # 49 tests del engine
npm run simular                     # docs/datos-simulacion.json
python docs/generar_doc_seccion2.py # este PDF
python docs/generar_metricas_seccion2.py
python docs/generar_autoaprendizaje_seccion2.py
```